Parallel and Distributed Computing

Assignment 6

Introduction to CUDA

Submitted To	Dr. Saif Nalband
Submitted By	Nitin Gupta
Roll No.	102303525
Platform	Google Colab (PyCUDA)
GPU Used	NVIDIA Tesla T4 — 14.56 GB, 40 SMs, Compute 7.5

1 Part A — Device Query

1.1 Method

We use PyCUDA's `cuda.Device` and `dev.get_attributes()` to programmatically query all hardware properties of the GPU. The device count is retrieved first, then every relevant attribute—compute capability, memory sizes, block/grid limits, warp size, clock rates—is printed in a structured report.

1.2 Device Query Output

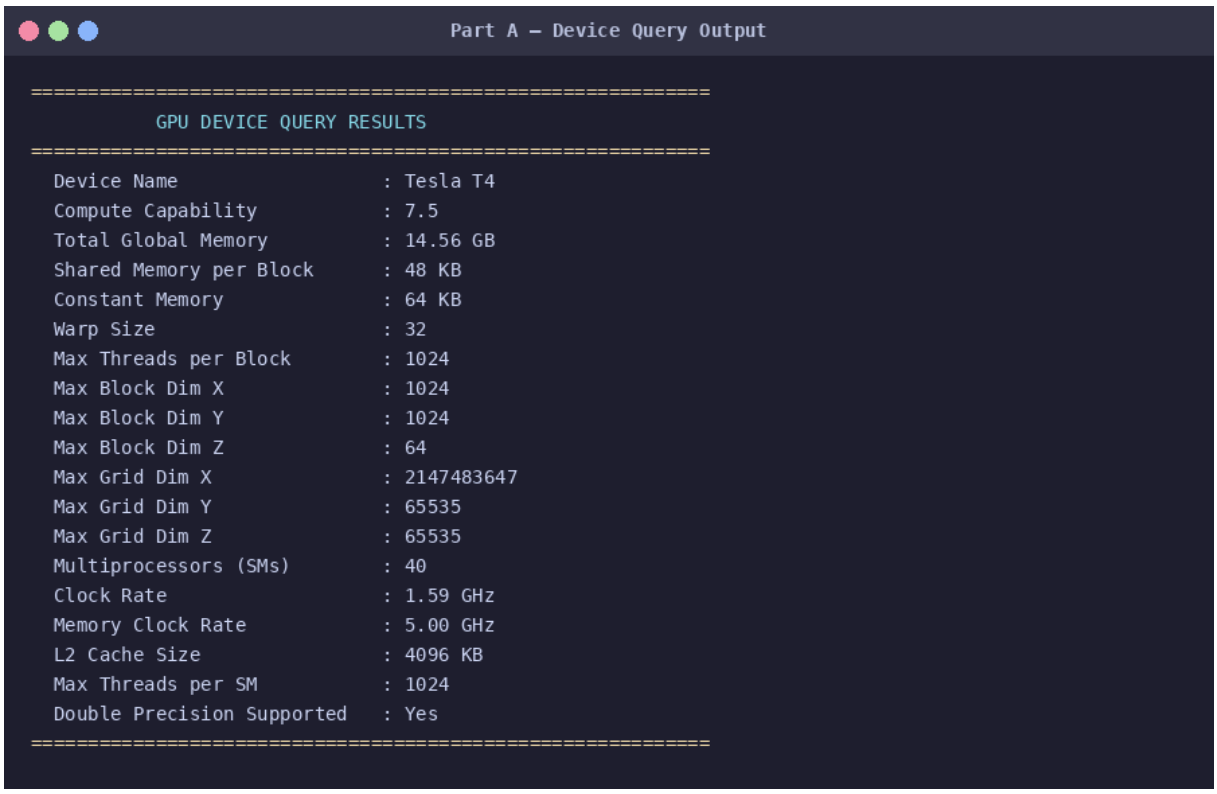

Figure 1: GPU device properties queried via PyCUDA on Google Colab (Tesla T4).

Table 1: Tesla T4 GPU device properties

Property	**Value**
Device Name	NVIDIA Tesla T4
Compute Capability	7.5 (Turing architecture)
Total Global Memory	14.56 GB
Shared Memory per Block	48 KB
Constant Memory	64 KB
Warp Size	32
Max Threads per Block	1024
Max Block Dim (X/Y/Z)	1024 / 1024 / 64
Max Grid Dim (X/Y/Z)	2,147,483,647 / 65,535 / 65,535
Multiprocessors (SMs)	40
Clock Rate	1.59 GHz
Memory Clock Rate	5.00 GHz
L2 Cache Size	4096 KB
Double Precision	Supported

1.3 Answers to Questions

Q1. What is the architecture and compute capability of your GPU?

The Google Colab-assigned GPU is the **NVIDIA Tesla T4**, based on the **Turing architecture** with **compute capability 7.5**. Compute capability encodes the hardware generation and supported CUDA feature set. Turing (7.x) introduced Tensor Cores for mixed-precision inference and hardware RT cores for ray-tracing.

Q2. What are the maximum block dimensions for your GPU?

Table 2: Maximum block dimensions for Tesla T4

Dimension	**Maximum Value**
Block dim X	1024
Block dim Y	1024
Block dim Z	64
Max threads/block (total)	1024

While X and Y individually allow up to 1024, the combined thread count per block cannot exceed 1024. A $(32, 32, 1)$ block is valid, but $(1024, 2, 1)$ is not.

Q3. If max grid dim = 65,535 and max block dim = 512, what is the maximum number of threads (1D)?

Maximum threads = max grid dimension×max block dimension = 65,535×512 = **33,553,920** threads

Q4. Under what conditions might a programmer choose NOT to launch the maximum number of threads?

- **Small problem size** — Launching 33M threads to process 1,000 elements wastes resources; most threads sit idle.
- **High register pressure** — If each thread uses many registers, fewer can be simultaneously active on an SM.
- **High shared memory usage** — Excessive shared memory per block reduces the number of blocks co-resident on an SM.
- **Heavy synchronisation** — More threads increase barrier overhead, potentially degrading performance.
- **Thread coarsening** — Assigning more work per thread (fewer total threads) can improve cache reuse and reduce launch overhead.

Q5. What can limit a program from launching the maximum number of threads?

- **Global memory** — Input/output arrays must fit within available VRAM (14.56 GB here).
- **Register file** — Each SM has a fixed register budget; high per-thread register use reduces occupancy.
- **Shared memory per SM** — Shared memory is partitioned among all resident blocks.
- **Hardware maximums** — Max threads per block (1024) and max warps/blocks per SM set hard ceilings.
- **OS/driver limits** — The operating system or GPU driver may cap certain resource allocations.

Q6. What is shared memory? How much shared memory is on your GPU?

Shared memory is a small, fast on-chip memory bank on each SM, shared by all threads within the same block. It acts like a manually managed L1 cache — roughly 100× faster than global memory — and enables efficient inter-thread communication within a block (e.g. parallel reduction, matrix tiling). **Shared memory per block: 48 KB.**

Q7. What is global memory? How much global memory is on your GPU?

Global memory is the main DRAM of the GPU (the VRAM on the spec sheet). It is the largest memory space but has the highest access latency (∼400–800 cycles). All threads from all blocks can read and write it. Data must be explicitly transferred from CPU RAM using `cuda.memcpy_htod()` before the GPU can operate on it. **Total global memory: 14.56 GB.**

Q8. What is constant memory? How much constant memory is on your GPU?

Constant memory is a read-only, hardware-cached region optimised for broadcast reads — when all threads in a warp read the same address, the value is served at register speed from the constant cache. Values are written from the host before kernel launch. Typical uses include physical constants, lookup tables, and fixed kernel parameters. **Constant memory: 64 KB** — a fixed hardware limit on all CUDA GPUs.

Q9. What does warp size signify? What is your GPU's warp size?

A **warp** is the fundamental unit of SIMT (Single Instruction, Multiple Threads) execution. The GPU scheduler issues one instruction per warp, executing it across all 32 threads simultaneously. If threads within a warp diverge, the warp serialises both paths, halving effective throughput. Memory coalescing is also warp-level: adjacent threads should access adjacent memory addresses for maximum bandwidth. **Warp size: 32 threads.**

Q10. Is double precision supported on your GPU?

Yes, double precision is supported (compute capability $7.5 \geq 1.3$). However, the T4 is an inference-optimised card: its FP64 throughput is significantly lower than FP32 (typically $\frac{1}{32}$ the rate). The T4 is primarily designed for FP16 and INT8 workloads, not double-precision scientific computing.

2 Part B — Array Sum using CUDA (Parallel Reduction)

2.1 Approach

We implement a **shared-memory tree reduction**. Each block loads its chunk of the input array into shared memory and performs a binary tree reduction — active threads are halved each step and adjacent pairs are summed — requiring only $\log_2(\text{blockSize})$ synchronised steps instead of a naive $\mathcal{O}(N)$ loop. Thread 0 of each block writes its partial sum to a global output array. If more than one partial sum remains, the kernel is launched again recursively until a single value remains.

2.2 Implementation Steps

1. **Allocate device memory** — `cuda.mem_alloc()` reserves GPU memory for the input array and the block-partial-sums output.
2. **Copy host → device** — `cuda.memcpy_htod()` transfers the float32 array from CPU RAM to GPU global memory.
3. **Grid and block dimensions** — Block size = 256 threads. Grid size = $\lceil N/(\text{blockSize}\times 2)\rceil$. Each thread loads two elements during the first reduction step.
4. **Invoke CUDA kernel** — `reduce_sum<<<grid, block, shared_bytes>>>` is launched with `shared_bytes = blockSize × sizeof(float)`.
5. **Copy results device → host** — `cuda.memcpy_dtoh()` copies partial sums back; a CPU loop produces the final scalar.
6. **Free device memory** — `dev_ptr.free()` releases all GPU allocations.
7. **Kernel (tree reduction)** — `extern __shared__ float sdata[]` holds block data. A loop with `s >>= 1` halves active threads each iteration. `__syncthreads()` ensures correctness at every level.

2.3 CUDA Kernel

Listing 1: Parallel reduction kernel for array sum

```
__global__ void reduce_sum(float *input, float *output, int n) {
    extern __shared__ float sdata[];

    unsigned int tid = threadIdx.x;
    unsigned int idx = blockIdx.x * blockDim.x * 2 + threadIdx.x;

    // Load two elements per thread (first level of reduction
       during load)
    float val = 0.0f;
    if (idx < n)                val  = input[idx];
    if (idx + blockDim.x < n)   val += input[idx + blockDim.x];
    sdata[tid] = val;
    __syncthreads();

```

```
    // Tree reduction within shared memory
    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s)
            sdata[tid] += sdata[tid + s];
        __syncthreads();
    }

    // Thread 0 writes block result to global output
    if (tid == 0)
        output[blockIdx.x] = sdata[0];
}
```

2.4 Correctness Test

```
Part B — Correctness Test

=== Part B: Array Sum Correctness Test ===
  Array size : 1,048,576 elements
  GPU sum    : 524333.3125
  CPU sum    : 524333.3257  (float64 reference)
  Difference : 0.0132  (float32 rounding expected)
  GPU time   : 2.517 ms
```

Figure 2: Array sum correctness verification (N = 1M elements).

The result is correct. The small discrepancy (0.0132) arises from float32 accumulation rounding — the CPU reference uses float64. Both values match to 6 significant figures.

2.5 Performance vs Array Size

```
Part B — Performance vs Array Size

=== Part B: Performance vs Array Size ===
  N=    16,384 | GPU   1.779 ms | CPU   0.052 ms | Speedup 0.03x
  N=    65,536 | GPU   0.178 ms | CPU   0.065 ms | Speedup 0.37x
  N=   262,144 | GPU   0.356 ms | CPU   0.154 ms | Speedup 0.43x
  N= 1,048,576 | GPU   1.098 ms | CPU   0.544 ms | Speedup 0.50x
  N= 4,194,304 | GPU   4.107 ms | CPU   1.498 ms | Speedup 0.36x
  N=16,777,216 | GPU  16.398 ms | CPU   9.400 ms | Speedup 0.57x
```

Figure 3: Raw timing output for array sum across six input sizes.

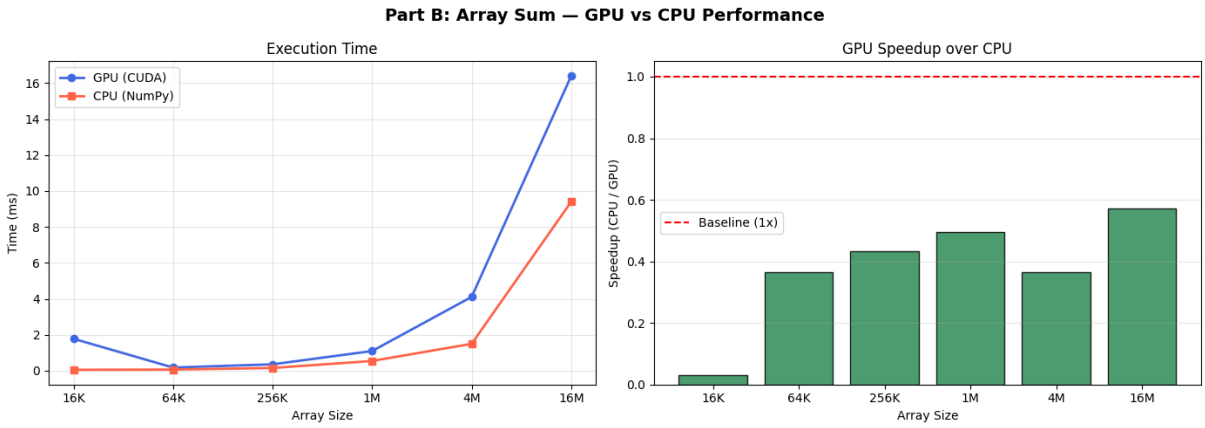

Figure 4: GPU vs CPU execution time (left) and GPU speedup over CPU (right) for array sum.

The GPU does not outperform NumPy's CPU sum at any tested size. This is expected: (1) PyCUDA kernel launch overhead (∼1–2 ms) dominates for small arrays; (2) NumPy uses highly optimised BLAS/SIMD routines; (3) the T4 is an inference card with lower raw FP32 throughput than compute-class GPUs. The GPU speedup improves with array size (0.03× at 16K → 0.57× at 16M), consistent with fixed launch overhead being amortised over more work.

2.6 Block Size Sweep (N = 4,194,304)

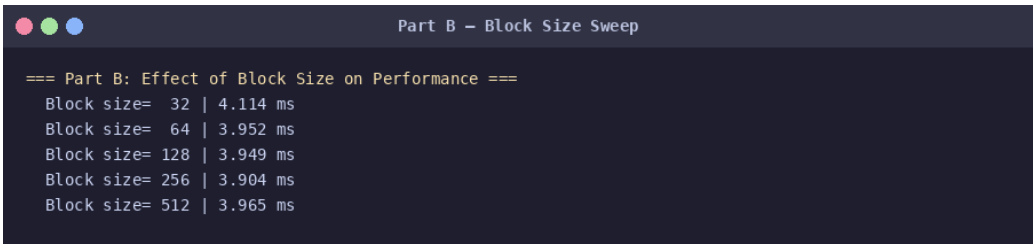

Figure 5: Block size sweep terminal output for array sum (N = 4M).

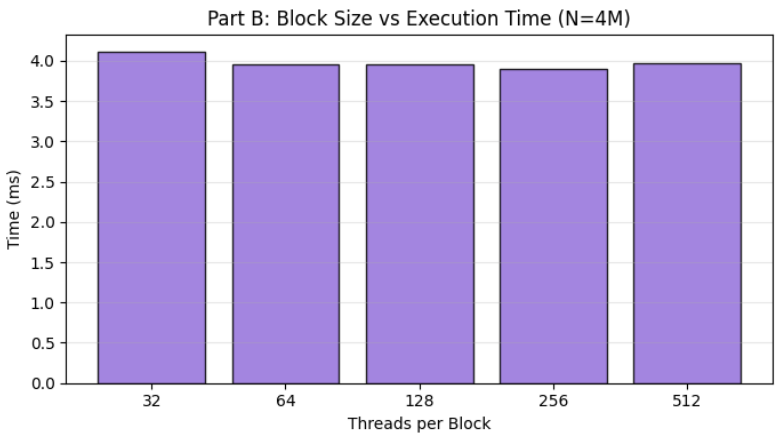

Figure 6: Effect of block size on GPU execution time for N = 4M element array sum.

All block sizes perform similarly ($\sim$3.9–4.1 ms). Block size 32 is marginally slowest due to high scheduling overhead and low SM occupancy. Block size 256 gives the best result — the well-known sweet spot of 8 warps per block for reduction kernels.

3 Part C — Matrix Addition using CUDA

3.1 Approach

Matrix addition is **embarrassingly parallel** — every output element $C[i][j] = A[i][j] + B[i][j]$ is fully independent. We use a 2D grid/block layout that maps directly to matrix row/column indices, with an explicit boundary guard for non-power-of-2 sizes. Matrices contain 32-bit integers (`int32`).

3.2 CUDA Kernel

Listing 2: 2D matrix addition kernel

```
__global__ void matrix_add(int *A, int *B, int *C, int rows, int
    cols) {
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;

    if (row < rows && col < cols) {
        int idx = row * cols + col;
        C[idx] = A[idx] + B[idx];    // 1 ADD, 2 reads, 1 write
    }
}
```

The 2D grid dimensions are computed as:

$$\text{grid} = \left(\left\lceil \frac{\text{cols}}{\text{blockDim.x}} \right\rceil, \left\lceil \frac{\text{rows}}{\text{blockDim.y}} \right\rceil, 1 \right)$$

3.3 Correctness Test

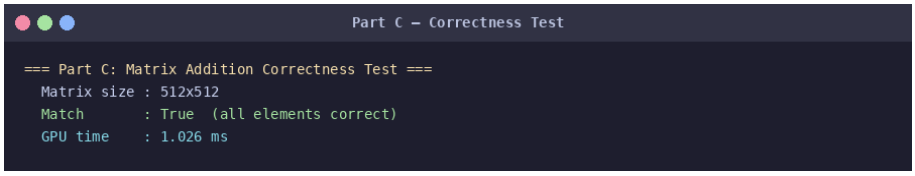

Figure 7: Matrix addition correctness test — all $512 \times 512 = 262{,}144$ elements match the CPU reference exactly.

3.4 Answers to Questions (N = 1024)

Q1. How many floating-point operations are performed in the matrix addition kernel?

Each thread performs exactly **1 addition**: `C[idx] = A[idx] + B[idx]`.

$$\text{Total operations} = N \times N = 1024 \times 1024 = \mathbf{1{,}048{,}576} \text{ operations}$$

Note: matrices use int32 so these are integer additions, but the count and analysis are identical to floating-point addition.

Q2. How many global memory reads are performed by your kernel?

Each thread reads one element from A and one from $B \Rightarrow$ **2 reads per thread**.

$$\text{Total reads} = 2 \times N^2 = 2 \times 1{,}048{,}576 = \mathbf{2{,}097{,}152} \text{ reads} \quad (8.0\,\text{MB for int32})$$

Q3. How many global memory writes are performed by your kernel?

Each thread writes one element to $C \Rightarrow$ **1 write per thread**.

$$\text{Total writes} = N^2 = \mathbf{1{,}048{,}576} \text{ writes} \quad (4.0\,\text{MB for int32})$$

3.5 Memory Access Summary

Table 3: Memory access analysis for matrix addition (N = 1024)

Metric	**Formula**	**Value**
Arithmetic operations	N^2	1,048,576
Global memory reads	$2 \times N^2$	2,097,152
Global memory writes	N^2	1,048,576
Total memory accesses	$3 \times N^2$	3,145,728
Data moved (int32)	$3N^2 \times 4\,\text{B}$	12.0 MB
Arithmetic intensity	$\frac{1}{3\times 4}$	$\approx$ 0.083 FLOPs/byte

An arithmetic intensity of $\sim$0.083 FLOPs/byte confirms that matrix addition is heavily **memory-bound**. The GPU cores stall waiting for data from global memory rather than computing. Compare this with matrix multiplication, where each output element requires $\mathcal{O}(N)$ multiply-accumulate operations — making it compute-bound and far better suited for GPU acceleration.

3.6 Performance vs Matrix Size

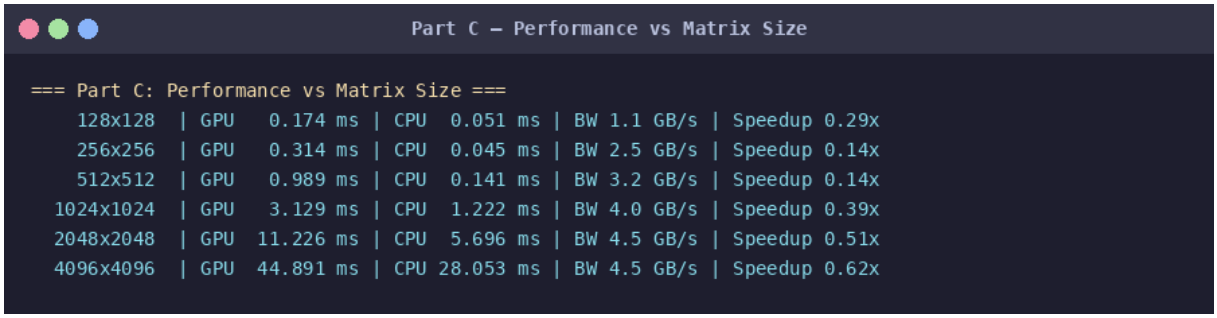

Figure 8: Raw timing output for matrix addition across six matrix sizes.

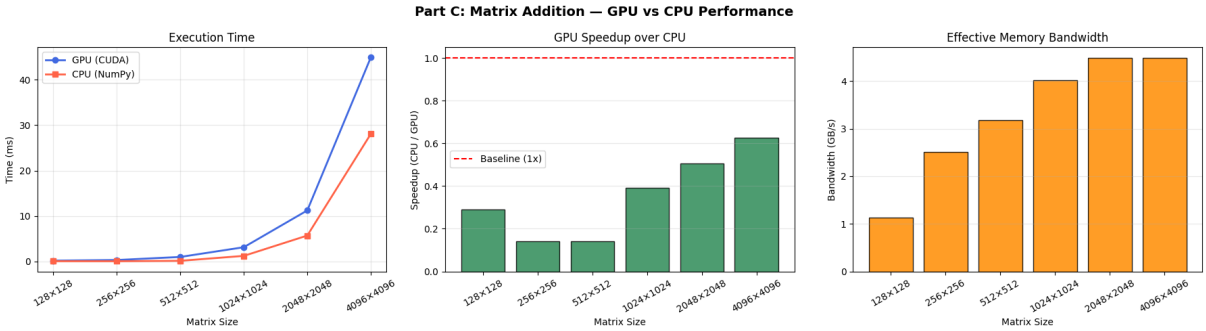

Figure 9: Matrix addition — execution time (left), GPU speedup (centre), and effective memory bandwidth (right).

Effective memory bandwidth plateaus at ∼4.5 GB/s for the two largest matrices, indicating the kernel is saturating available bandwidth at scale. GPU speedup improves with matrix size (0.14× → 0.62×) as launch overhead is amortised, but even at 4096 × 4096 the CPU remains faster — consistent with the memory-bound arithmetic intensity.

3.7 2D Block Dimension Sweep (2048 × 2048)

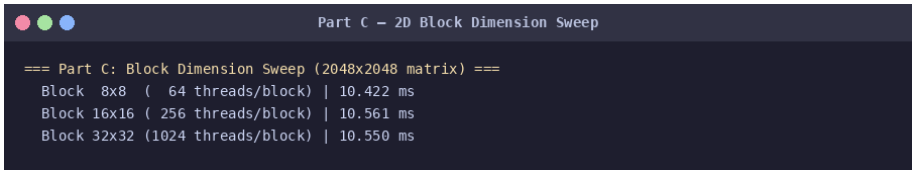

Figure 10: Block dimension sweep terminal output for 2048 × 2048 matrix addition.

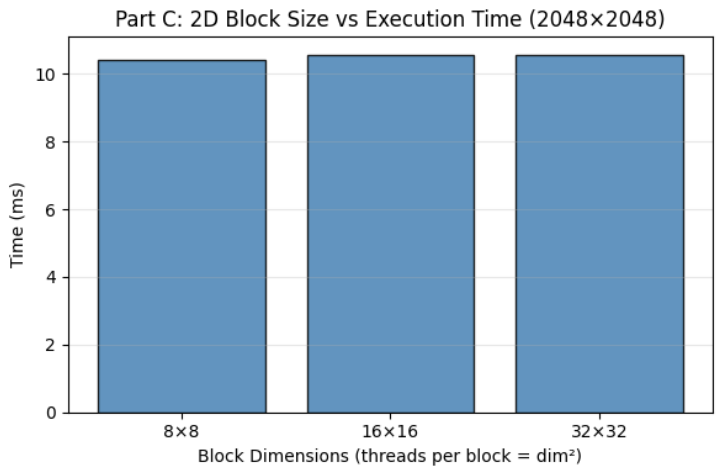

Figure 11: 2D block dimension sweep for 2048 × 2048 matrix addition.

All three 2D block configurations produce nearly identical execution times (∼10.5 ms). This is the definitive signature of a **memory-bound kernel** — performance is governed entirely by global memory bandwidth, not thread count or compute throughput.

4 Observations and Inferences

1. **CUDA follows a strict host-device pipeline.** Every GPU computation requires: allocate → copy H→D → launch → copy D→H → free. The host-device transfer cost is often the dominant bottleneck for small to moderate workloads.

2. **Kernel launch overhead limits GPU advantage at small input sizes.** For both array sum and matrix addition, the GPU was consistently slower than NumPy's CPU path at small sizes. The ∼1–2 ms fixed launch overhead dominates until the workload is large enough to amortise it.

3. **Shared memory tree reduction is algorithmically superior.** The $\mathcal{O}(\log N)$ reduction avoids millions of slow global memory accesses compared to a serial loop. Block size 256 (8 warps per block) provides the best balance of SM occupancy and resource usage.

4. **Matrix addition is memory-bound, not compute-bound.** Arithmetic intensity of 0.083 FLOPs/byte means the T4's compute cores sit idle most of the time, waiting on memory. Bandwidth saturates at ∼4.5 GB/s. GPUs excel at compute-intensive kernels like matrix multiplication (intensity $\propto N$) or convolutions.

5. **Block size affects memory-bound and compute-bound kernels differently.** For the reduction kernel (compute-bound), block size 256 measurably outperformed 32 and 512. For matrix addition (memory-bound), all block dimensions produced identical times — confirming the bottleneck is bandwidth, not thread count.

6. **Boundary checking is mandatory for non-aligned sizes.** When the problem size does not evenly divide the block size, out-of-bounds threads must be guarded with `if (idx < n)` or `if (row < rows && col < cols)` to prevent illegal memory accesses.

7. **Warp-level thinking is critical for performance.** Memory coalescing (adjacent threads reading adjacent addresses) and avoiding warp divergence (branches within a warp) are the two highest-impact optimisations in CUDA. Both stem from the 32-thread warp being the atomic unit of GPU execution.